

Gazetteer & Source Workflow

Pre-discussion briefing — Baserow, the public entry points, and WHG integration

Prepared by: Stephen Gadd · **Date:** 19 June 2026 · **For:** Ruth Mostern, Palak Vashist, Alexandra Straub · **Purpose:** circulated ahead of today's discussion

We now have the makings of a system for tracking *leads on sources we might index* (especially print historical gazetteers and community recommendations) and for *managing the editorial workflow* around incoming gazetteers. Palak has built this in **Baserow**, and I've connected it to the website. Three questions have surfaced that need a team decision rather than a technical one. This note frames each, with a recommendation, so we can decide efficiently today.

A note on terms. Going forward, every contribution to WHG is a **gazetteer** — the single term that replaces the older "dataset" / "collection" split. Today a gazetteer is realised in the software as a *Dataset* or a *Collection*; in future the concept is intended to encompass routes, itineraries and networks as well. I've used "gazetteer" throughout. To avoid confusion: a *print historical gazetteer* is a *source* we might turn *into* a WHG gazetteer — it is a lead, not yet a gazetteer in the platform sense.

Where we are now

- **Baserow workspace (Palak)** — an open-source, free-tier alternative to Airtable, with tables for incoming gazetteers, projects and submitters, a public submission form, and an email notification when something arrives.
- **Website link is live** SHIPPED — the site links out to Palak's public form. The integration is deliberately *thin* (a simple link), which keeps our options open and our lock-in low.
- **Interim relabel** SHIPPED — the menu item now reads "*Suggest a Source*" rather than "Submit a Dataset" (see Question 1 for why).
- We *retired* an earlier prototype that built this inside WHG's own software, in favour of Baserow's no-code approach — so Palak and colleagues can adjust fields and views without a developer.

Question 1 — Two doors: "Suggest a Source" vs contributing a gazetteer

The website already has a self-service route at </datasets/validate/>: that is for someone who *has* a *gazetteer* (today, a Dataset or Collection) and wants to upload and publish it to WHG. Palak's Baserow form is for something different — **capturing leads and suggestions about sources we might index** (a print gazetteer PDF, a "you should look at X" recommendation), which Palak then triages and which may eventually *become* a gazetteer.

The friction: the Baserow form is titled "*Submit Your Dataset to WHG*" and the website link originally said "*Submit a Dataset*" — which reads as a **second, competing upload route**. Someone who has already contributed a gazetteer through the validate flow would see two "submit" doors. The form's fields (title, stage-of-data, link to files) also lean toward "your finished gazetteer," which deepens the overlap. There is also a separate, real need to keep track of **traditional gazetteer submissions** — so the two purposes genuinely coexist and must be kept legible.

Recommendation: frame the Baserow form explicitly as *suggesting a source*, clearly distinct from contributing a finished gazetteer. Present two clearly-separated doors on the site — “*Contribute a gazetteer*” → validate, and “*Suggest a source*” → Baserow — and have Palak retitle the form and reframe its fields toward *sources / suggestions* so a print gazetteer or recommendation fits naturally.

Decision needed: Is the Baserow form (a) a “*suggest a source we should consider*” tool — leads, distinct from contribution; or (b) a *general front door* that triages everyone and routes gazetteer-owners onward to the validate flow? The wording, fields and site layout all follow from this.

Question 2 — Mirroring our gazetteers into Baserow

A natural next step was proposed: push our internal gazetteers (currently modelled in the software as Datasets and Collections) into Baserow — backfilling the current state, then updating Baserow whenever those records change — so the team can manage everything in one place. This is feasible and useful, but it deepens the integration, so a few design points matter.

The safe shape

- **One-way mirror.** WHG remains the single source of truth and pushes canonical fields *out* to Baserow, which acts as a dashboard / workflow overlay. If Baserow ever went away, WHG would be unaffected — that is what makes the deeper link acceptable.
- **Resilient and in the background.** The push must never block or fail a WHG save; a Baserow outage should be invisible to our users. (Technically: queued, retried, off the request path.)
- **Don't overwrite Palak's columns.** Baserow's gazetteer table will hold *editorial* fields Palak edits by hand (status, assignee, notes). The sync must only write the fields WHG owns and must never touch those workflow columns.
- **Backfill first.** Start with a one-off load of all current gazetteers (in a preview / dry-run mode so we can check the mapping in Baserow), and only then switch on the live updates.
- **Token hygiene.** This needs Baserow's full read/write API key, stored as a secret on our side and — since it was shared by email in the clear — **rotated first**.

Recommendation: proceed, but as a strictly one-way mirror with field-level boundaries, built backfill-first with a preview step. Keep *mirrored real gazetteers* and *suggested sources* (Question 1) as distinct tables/views so the two don't blur.

Decisions needed: (a) which fields should mirror, and which gazetteer type comes first — Datasets or Collections? (b) confirmation that Baserow's gazetteer table is a mirror + workflow overlay, *not* a second source of truth, and which columns are Palak's editorial ones to protect.

Question 3 — Relying on Baserow: cost, self-hosting, and “redundancy”

A fair concern was raised: Baserow introduces an external dependency that may not always be free, and — since it is itself built on the same Django technology we already run — it looks like it might duplicate what we have.

Three things are worth separating:

- **"It may not stay free."** True of the hosted *Cloud* tier. But Baserow's *self-hosted* edition is genuinely open-source and free, so the cost risk has a built-in escape hatch. Because we kept the integration to a thin link, the switching cost is low (an export/import).
- **"It runs on Django — redundancy."** The important nuance: Baserow is a *separate product*, not a component we can fold into the WHG application. Self-hosting it means *operating a second service* (its own database, upgrades, backups, security) alongside WHG — we don't get to reuse our existing system. The redundancy is real at the infrastructure level; the upside is that running it is at least familiar territory.
- **Data residency.** On Cloud, the data lives off-Pitt; self-hosting keeps it on our own servers.

Option	Strengths	Costs / risks
Baserow Cloud (current)	Zero operations; instant; Palak self-serves fields & forms.	Possible future cost; data held off-Pitt.
Self-host Baserow	Free; data stays at Pitt; no vendor cost risk.	A second full stack for us to run, patch and back up.
Build inside WHG (the retired approach)	No new stack; tight integration with our data.	Developer time; Palak could not adjust it herself.

Recommendation: stay on Baserow Cloud for now (zero operations while the workflow is still taking shape), keep the integration thin, and treat self-hosting as the fallback we reach for only if cost or data-residency actually becomes a problem. I would *not* rebuild this inside WHG — the team specifically wanted no-code self-service, which is the one thing our own software cannot cheaply provide.

Decision needed: Are we comfortable starting on Cloud, with self-hosting held in reserve? If data residency is a firm requirement now, we self-host from the outset instead.

Summary — decisions for today

Decision	Options	My recommendation
1. Purpose of the Baserow form	(a) Suggest-a-source / leads, distinct from contribution; or (b) general front door that triages to validate.	(a) — and retitle the form + present two clear doors.
2a. Fields & types to mirror	Define the field list, and whether Datasets or Collections come first.	Datasets first; agree a short canonical field list.
2b. Source of truth & protected columns	Confirm WHG is authoritative; list Palak's editorial columns.	One-way mirror; never overwrite editorial columns.
3. Hosting Baserow	Cloud now / self-host now / build in-house.	Cloud now; self-host in reserve; don't build in-house.

Once Questions 1 and 2 are settled, the remaining work is quick: the website wording follows directly from Question 1, and I can build the gazetteer backfill in a safe preview mode for review before anything is written to Baserow.